

```
[93]: import sys
!{sys.executable} -m pip install seaborn scikit-learn matplotlib

Requirement already satisfied: seaborn in /home/cloud/anaconda3/lib/python3.12/site-packages (0.13.2)
Requirement already satisfied: scikit-learn in /home/cloud/anaconda3/lib/python3.12/site-packages (1.8.0)
Requirement already satisfied: matplotlib in /home/cloud/anaconda3/lib/python3.12/site-packages (3.10.1)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /home/cloud/anaconda3/lib/python3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /home/cloud/anaconda3/lib/python3.12/site-packages (from seaborn) (2.2.3)
Requirement already satisfied: scipy>=1.10.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.2.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (11.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /home/cloud/anaconda3/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /home/cloud/anaconda3/lib/python3.12/site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in /home/cloud/anaconda3/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

Utility Functions

```
[99]: def rmsle(y_true, y_pred):
    """Root Mean Squared Logarithmic Error.

    Clamps predictions to >= 0 before computing loglp to avoid NaN.
    """
    y_pred = np.maximum(y_pred, 0)
    return np.sqrt(np.mean((np.loglp(y_pred) - np.loglp(y_true)) ** 2))

def clamp(predictions):
    """Clamp predictions to non-negative values (bike counts can't be negative)."""
    return np.maximum(predictions, 0)

def engineer_features(df):
    """Extract temporal and interaction features from a bike-sharing DataFrame.

    Creates: hour, weekday, month, year, rush_hour, night, weekend,
            temp_sq, hum_sq, temp_x_hour.
    """
    df = df.copy()
    df['datetime'] = pd.to_datetime(df['datetime'], format='mixed', dayfirst=True)

    # Temporal features
    df['hour'] = df['datetime'].dt.hour
    df['weekday'] = df['datetime'].dt.weekday
    df['month'] = df['datetime'].dt.month
    df['year'] = df['datetime'].dt.year

    # Binary indicators (vectorized - no slow .apply(lambda))
    df['rush_hour'] = df['hour'].isin([7, 8, 9, 17, 18, 19]).astype(int)
    df['night'] = ((df['hour'] < 6) | (df['hour'] >= 22)).astype(int)
    df['weekend'] = (df['weekday'] >= 5).astype(int)

    # Nonlinear & interaction terms
    df['temp_sq'] = df['temp'] ** 2
    df['hum_sq'] = df['humidity'] ** 2
    df['temp_x_hour'] = df['temp'] * df['hour']

    return df

def tune_model(model_cls, alphas, X_train, y_train, X_val, y_val,
               log_target=False, **model_kwargs):
    """Evaluate a regularized model across a grid of alpha values.

    Parameters
    -----
    model_cls : sklearn estimator class (Ridge or Lasso)
    alphas : list of float - regularization strengths to try
    log_target : if True, train on loglp(y) and inverse-transform predictions
    model_kwargs : extra kwargs forwarded to model_cls (e.g. max_iter)

    Returns
    -----
    results : list of (alpha, rmsle_score) tuples
    """
    y_fit = np.loglp(y_train) if log_target else y_train
    results = []
    for alpha in alphas:
        model = model_cls(alpha=alpha, **model_kwargs).fit(X_train, y_fit)
        pred = model.predict(X_val)
        if log_target:
            pred = np.exp(pred)
```

```

    if log_target:
        pred = np.exp(pred)
        score = rmsle(y_val, clamp(pred))
        results.append((alpha, score))
        print(f" alpha={alpha:<6} RMSLE={score:.4f}")
    return results

```

Load Data & Feature Engineering (single pass)

```

[102]: # Load once, engineer once – no redundant I/O or parsing
train = engineer_features(pd.read_csv('bike_train.csv'))
test = engineer_features(pd.read_csv('bike_test.csv'))

```

```

FEATURES = [
    'season', 'holiday', 'workingday', 'weather',
    'temp', 'atemp', 'humidity', 'windspeed',
    'hour', 'weekday', 'month', 'year',
    'rush_hour', 'night', 'weekend',
    'temp_sq', 'hum_sq', 'temp_x_hour',
]

```

```

print(f"Train shape: {train.shape}")
print(f"Test shape: {test.shape}")
print(f"Features ({len(FEATURES)}): {FEATURES}")

```

```

Train shape: (10450, 22)
Test shape: (2613, 19)
Features (18): ['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'hour', 'weekday', 'month', 'year', 'rush_hour', 'night', 'weekend', 'temp_sq', 'hum_sq', 'temp_x_hour']

```

Q1 — Examine Dataset Size, Missing Values, and Feature Types

```

[105]: print("=== Feature Types ===")
print(train.dtypes)
print("\n=== Missing Values (Train) ===")
print(train.isnull().sum())
print("\n=== Missing Values (Test) ===")
print(test.isnull().sum())
print("\n=== Basic Statistics ===")
train.describe()

```

```

=== Feature Types ===
datetime      datetime64[ns]
season         int64
holiday        int64
workingday     int64
weather        int64
temp           float64
atemp          float64
humidity       int64
windspeed      float64
casual         int64
registered     int64
count         int64
hour           int32
weekday        int32
month          int32
year           int32
rush_hour      int64
night          int64
weekend        int64
temp_sq        float64
hum_sq         int64
temp_x_hour    float64
dtype: object

```

=== Missing Values (Train) ===

```
datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
casual      0
registered  0
count       0
hour        0
weekday     0
month       0
year        0
rush_hour   0
night       0
weekend     0
temp_sq     0
hum_sq      0
temp_x_hour 0
dtype: int64
```

=== Missing Values (Test) ===

```
datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
hour        0
weekday     0
month       0
year        0
rush_hour   0
night       0
weekend     0
temp_sq     0
hum_sq      0
temp_x_hour 0
dtype: int64
```

=== Basic Statistics ===

[105]:	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	
count	10450	10450.000000	10450.000000	10450.000000	10450.000000	10450.000000	10450.000000	10450.000000	10450.000000	10450
mean	2011-12-26 18:06:50.826507008	2.507943	0.028804	0.675694	1.413876	20.191700	23.605793	61.924211	12.765259	35
min	2011-01-01 00:00:00	1.000000	0.000000	0.000000	1.000000	0.820000	0.760000	0.000000	0.000000	0
25%	2011-07-01 14:15:01.500000	2.000000	0.000000	0.000000	1.000000	13.940000	16.665000	47.000000	7.001500	4
50%	2012-01-01 08:30:00.500000	3.000000	0.000000	1.000000	1.000000	20.500000	24.240000	62.000000	12.450050	16
75%	2012-07-02 02:30:00.750000128	4.000000	0.000000	1.000000	2.000000	26.240000	31.060000	77.000000	16.997900	49
max	2012-12-19 23:00:00	4.000000	1.000000	1.000000	4.000000	41.000000	45.455000	100.000000	56.996900	367
std	NaN	1.116946	0.167263	0.468137	0.632258	7.792683	8.478045	19.245193	8.102821	49

8 rows × 22 columns

Observations:

- **10,450 training samples**, 9 features + 3 target-related columns (casual, registered, count).
- **2,613 test samples** with no target.
- **Zero missing values** in both train and test.
- Feature types: `datetime` is a string (needs parsing); `season`, `weather`, `holiday`, `workingday` are categorical integers; remaining are numerical floats/ints.

Q2 — Visualize Relationships Between Key Features and Count

```
[109]: fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle('Key Feature vs Count Relationships', fontsize=14, fontweight='bold')

# Hour
hour_avg = train.groupby('hour')['count'].mean()
axes[0, 0].plot(hour_avg.index, hour_avg.values, marker='o', color='steelblue')
axes[0, 0].set(title='Avg Rentals by Hour', xlabel='Hour', ylabel='Avg Count')

# Season
sns.boxplot(x='season', y='count', data=train, ax=axes[0, 1], palette='Set2')
axes[0, 1].set_title('Count by Season')

# Temperature
axes[0, 2].scatter(train['temp'], train['count'], alpha=0.3, s=5, color='coral')
axes[0, 2].set(title='Temp vs Count', xlabel='Temp (°C)')

# Weather
sns.boxplot(x='weather', y='count', data=train, ax=axes[1, 0], palette='Set3')
axes[1, 0].set_title('Count by Weather Condition')

# Weekday
weekday_avg = train.groupby('weekday')['count'].mean()
axes[1, 1].bar(weekday_avg.index, weekday_avg.values, color='mediumseagreen')
axes[1, 1].set_title('Avg Rentals by Weekday')
axes[1, 1].set_xticks(range(7))
axes[1, 1].set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])

# Humidity
axes[1, 2].scatter(train['humidity'], train['count'], alpha=0.3, s=5, color='mediumpurple')
axes[1, 2].set(title='Humidity vs Count', xlabel='Humidity (%)')

plt.tight_layout()
plt.show()
```

PreviewFileEditViewGoToolsWindowHelp

Screenshot 20...

8 rows x 22 columns

Observations:

- 16,430 training samples, 9 features + 3 target-related columns (casual, registered, count).
- 2,613 test samples with no target.
- Zero missing values in both train and test.
- Feature types: datetime is a string (needs parsing); season, weather, holiday, workingday are categorical integers; remaining are numerical float/integers.

Q2 — Visualize Relationships Between Key Features and Count

```
[108]: fig, axes = plt.subplots(2, 3, figsize=(16, 9))

# Hour
hour_avg = train.groupby('hour')['count'].mean()
axes[0, 0].plot(hour_avg.index, hour_avg.values, marker='o', color='steelblue')
axes[0, 0].set_title('Avg Rentals by Hour', xlabel='Hour', ylabel='Avg Count')

# Season
sns.boxplot(ax=axes[0, 1], data=train, y='count', x=season, palette='Set2')
axes[0, 1].set_title('Count by Season')

# Temperature
axes[0, 2].scatter(train['temp'], train['count'], alpha=0.3, s=5, color='coral')
axes[0, 2].set_title('Temp vs Count', xlabel='Temp (°C)')

# Weather
sns.boxplot(ax=axes[1, 0], data=train, y='count', x=weather, palette='Set3')
axes[1, 0].set_title('Count by Weather Condition')

# Weekday
weekday_avg = train.groupby('weekday')['count'].mean()
axes[1, 1].bar(weekday_avg.index, weekday_avg.values, color='mediumseagreen')
axes[1, 1].set_title('Avg Rentals by Weekday')
axes[1, 1].set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])

# Humidity
axes[1, 2].scatter(train['humidity'], train['count'], alpha=0.3, s=5, color='mediumpurple')
axes[1, 2].set_title('Humidity vs Count', xlabel='Humidity (%)')

plt.tight_layout()
plt.show()
```

on.ipynb ~/.../Bike...

CHATCODEX

pd.DataFrame.from_dict(results, o...

.venv (3.14.3) (Python 3.14.3)

/fix

Bike_Sharing_Assignment.ipynb - Cell 32:1-37

Copilot CLI

A Copilot CLI session has begun working on your request. Follow its progress in the sessions list.

6:37 -- Fo

7:07 296 KB PN

7:08 183 KB PN

7:08 167 KB PN

7:09 155 KB PN

7:09 163 KB PN

Python

zsh

Python [jup...

ML_Assignment1_Solution.ipynb - Cell 21

/

Copilot CLI

Default Approvals

Spaces: 4 LF

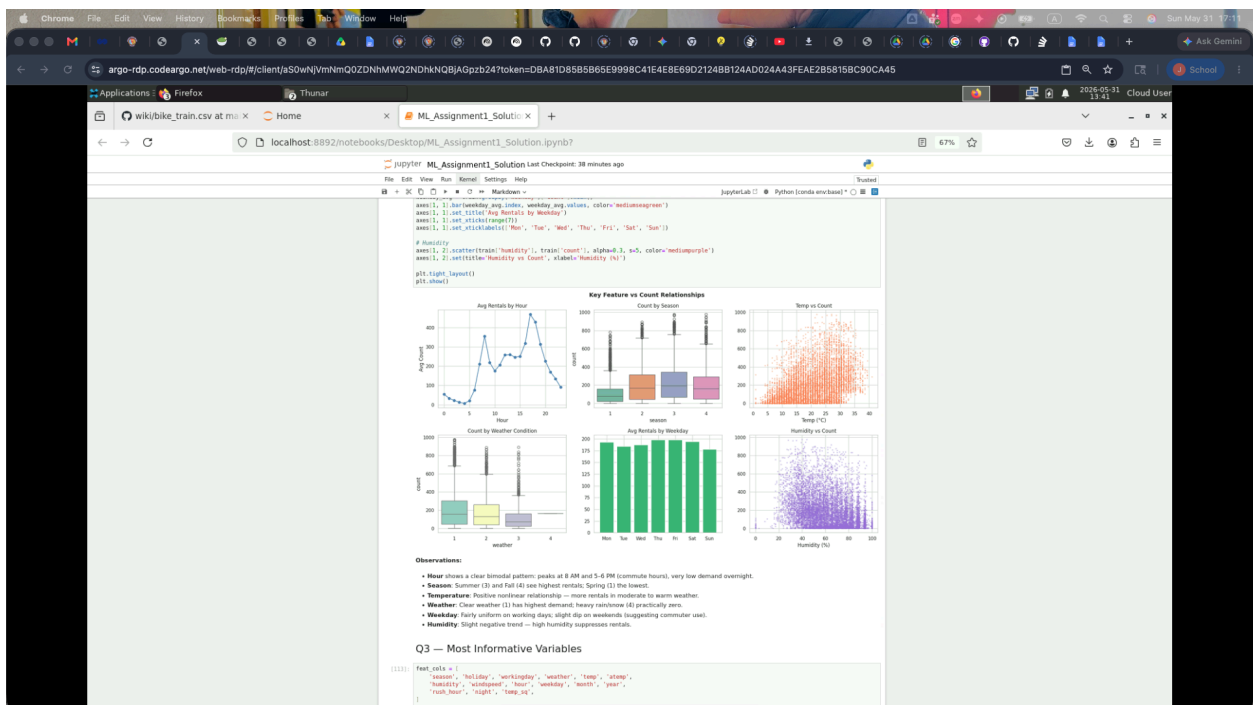
Cell 21 of 80

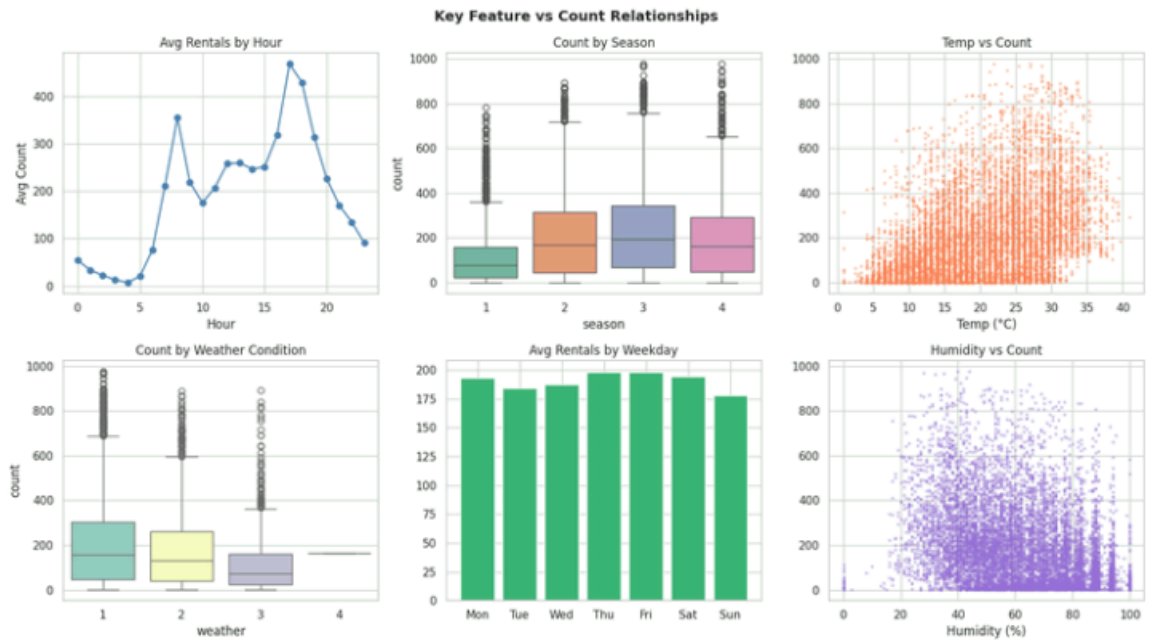
Final Submission

- Place all required files into a single ZIP folder:
 - ✓ .ipynb notebook
 - ✓ submission.csv
 - ✓ Screenshot
- Upload the ZIP file on the Taxila (e-learn portal) under Assignment 1.
- Late submissions will not be accepted.
- Ensure that the uploaded files are correct before final submission.

All the Best for Your Assignment!

Happy Learning 🌟





Observations:

- **Hour** shows a clear bimodal pattern: peaks at 8 AM and 5-6 PM (commute hours), very low demand overnight.
- **Season:** Summer (3) and Fall (4) see highest rentals; Spring (1) the lowest.
- **Temperature:** Positive nonlinear relationship — more rentals in moderate to warm weather.
- **Weather:** Clear weather (1) has highest demand; heavy rain/snow (4) practically zero.
- **Weekday:** Fairly uniform on working days; slight dip on weekends (suggesting commuter use).
- **Humidity:** Slight negative trend — high humidity suppresses rentals.

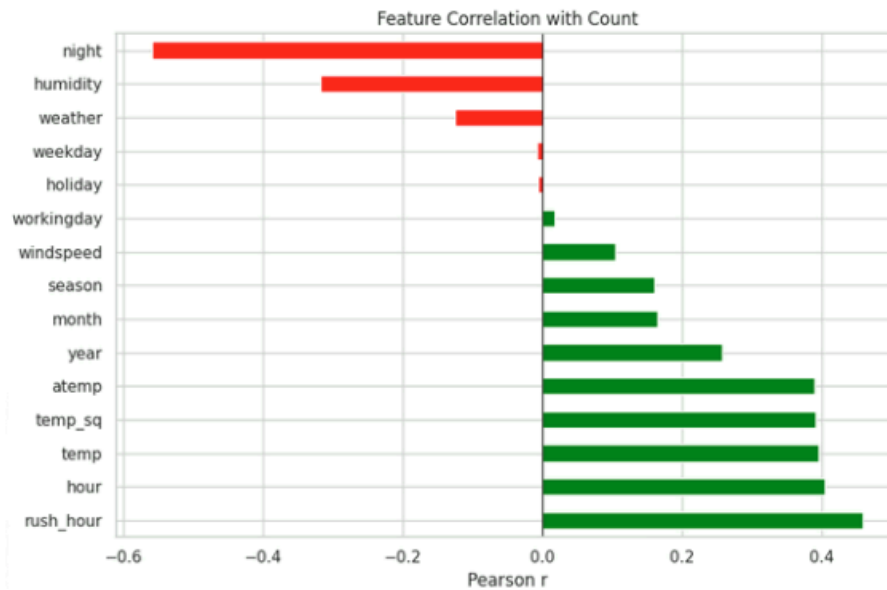
Q3 — Most Informative Variables

```
[113]: feat_cols = [
    'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp',
    'humidity', 'windspeed', 'hour', 'weekday', 'month', 'year',
    'rush_hour', 'night', 'temp_sq',
]

corr = train[feat_cols + ['count']].corr()['count'].drop('count').sort_values(ascending=False)

plt.figure(figsize=(9, 6))
colors = ['green' if v > 0 else 'red' for v in corr.values]
corr.plot(kind='barh', color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Feature Correlation with Count')
plt.xlabel('Pearson r')
plt.tight_layout()
plt.show()

print(corr)
```



```
rush_hour    0.459822
hour          0.404188
temp          0.396451
temp_sq       0.392137
atemp         0.390642
year          0.257987
month         0.165475
season        0.160333
windspeed     0.105318
workingday    0.017361
holiday       -0.005615
weekday       -0.006489
weather       -0.124402
humidity      -0.316607
night         -0.558948
Name: count, dtype: float64
```

Most Informative Variables (ranked by |correlation| with count):

Rank	Feature	Reason
1	hour	Bimodal commute pattern — strongest single predictor
2	temp / atemp	More riding in warm weather; nonlinear relationship
3	year	Demand grew from 2011 → 2012 (growth trend)
4	rush_hour	Engineered flag capturing peak commute demand
5	season / month	Seasonal patterns aligned with temperature
6	humidity	High humidity reduces comfort and ridership
7	weather	Severe weather drastically cuts demand
8	workingday	Differentiates commuter vs leisure patterns

windspeed and holiday have low individual correlation but may help in interaction terms.

Q4 — Feature Engineering (Optional)

Features engineered (applied in the data-loading cell above via `engineer_features()`):

- `hour`, `weekday`, `month`, `year` — extracted from `datetime`
- `rush_hour` — binary flag for morning/evening peak commute hours
- `night` — binary flag for overnight hours (demand ~0)
- `weekend` — differentiates commuter vs leisure patterns
- `temp_sq`, `hum_sq` — quadratic terms to capture nonlinear effects
- `temp_x_hour` — interaction: high temperature at peak hours boosts demand

Q5 — Train/Validation Split & Simple Linear Regression (1 Mark)

```
[68]: X = train[FEATURES]
      y = train['count']
      X_test_final = test[FEATURES]

      X_train, X_val, y_train, y_val = train_test_split(
          X, y, test_size=0.2, random_state=42
      )
      print(f"Train: {X_train.shape}, Val: {X_val.shape}")

      # Scale features
      scaler = StandardScaler()
      X_train_sc = scaler.fit_transform(X_train)
      X_val_sc = scaler.transform(X_val)
      X_test_sc = scaler.transform(X_test_final)

      # Simple Linear Regression
      lr = LinearRegression().fit(X_train_sc, y_train)
      lr_rmse = rmse(y_val, clamp(lr.predict(X_val_sc)))
      print(f"\nQ5 - Linear Regression RMSLE (validation): {lr_rmse:.4f}")
```

Train: (8360, 18), Val: (2090, 18)

Q5 - Linear Regression RMSLE (validation): 1.2475

Q6 — Polynomial Features + Ridge / Lasso (1 Mark)

```
[71]: # Degree-2 polynomial expansion
poly2 = PolynomialFeatures(degree=2, include_bias=False)
X_tr_p2 = poly2.fit_transform(X_train_sc)
X_va_p2 = poly2.transform(X_val_sc)
X_te_p2 = poly2.transform(X_test_sc)
print(f'Poly(2) feature dim: {X_tr_p2.shape[1]}')

# Collect all model results
results = {'Linear Regression': lr_rmsle}

# (a) Poly + plain Linear Regression
lr2 = LinearRegression().fit(X_tr_p2, y_train)
results['Poly(2) + Linear'] = rmsle(y_val, clamp(lr2.predict(X_va_p2)))

# (b) Ridge - alpha sweep
print("\nRidge alpha search:")
ridge_results = tune_model(
    Ridge, [0.1, 1, 10, 50, 100, 500, 1000],
    X_tr_p2, y_train, X_va_p2, y_val,
)
best_ridge_alpha = max(ridge_results, key=lambda x: -x[1])[0] # lowest RMSLE
best_ridge = Ridge(alpha=best_ridge_alpha).fit(X_tr_p2, y_train)
results[f'Poly(2)+Ridge(alpha={best_ridge_alpha})'] = rmsle(y_val, clamp(best_ridge.predict(X_va_p2)))

# (c) Lasso - alpha sweep
print("\nLasso alpha search:")
lasso_results = tune_model(
    Lasso, [0.001, 0.01, 0.1, 1, 10],
    X_tr_p2, y_train, X_va_p2, y_val,
    max_iter=5000,
)
best_lasso_alpha = max(lasso_results, key=lambda x: -x[1])[0]
best_lasso = Lasso(alpha=best_lasso_alpha, max_iter=5000).fit(X_tr_p2, y_train)
results[f'Poly(2)+Lasso(alpha={best_lasso_alpha})'] = rmsle(y_val, clamp(best_lasso.predict(X_va_p2)))

# (d) KEY TRICK: Ridge on log-transformed target
print("\nRidge (log-target) alpha search:")
log_ridge_results = tune_model(
    Ridge, [0.1, 1, 10, 100, 500],
    X_tr_p2, y_train, X_va_p2, y_val,
    log_target=True,
)
best_log_alpha = max(log_ridge_results, key=lambda x: -x[1])[0]
best_log_ridge = Ridge(alpha=best_log_alpha).fit(X_tr_p2, np.log1p(y_train))
log_pred = clamp(np.expml(best_log_ridge.predict(X_va_p2)))
results[f'Poly(2)+Ridge_LogTarget(alpha={best_log_alpha})'] = rmsle(y_val, log_pred)

print("\nAll model RMSLE scores collected!")
```

Poly(2) feature dim: 189

Ridge alpha search:

alpha=0.1	RMSLE=1.1136
alpha=1	RMSLE=1.1093
alpha=10	RMSLE=1.1008
alpha=50	RMSLE=1.0980
alpha=100	RMSLE=1.0916
alpha=500	RMSLE=1.0819
alpha=1000	RMSLE=1.0625

Lasso alpha search:

alpha=0.001	RMSLE=1.1118
alpha=0.01	RMSLE=1.1016
alpha=0.1	RMSLE=1.0925
alpha=1	RMSLE=1.0995
alpha=10	RMSLE=0.9333

Ridge (log-target) alpha search:

alpha=0.1	RMSLE=0.6380
alpha=1	RMSLE=0.6373
alpha=10	RMSLE=0.6367
alpha=100	RMSLE=0.6373
alpha=500	RMSLE=0.6415

All model RMSLE scores collected!

Q7 — Model Comparison Summary Table (1.5 Marks)

```
[73]: results_df = pd.DataFrame.from_dict(
    results, orient='index', columns=['RMSLE (Validation)'])
    ).sort_values('RMSLE (Validation)')

results_df['Rank'] = range(1, len(results_df) + 1)
results_df['Key Observation'] = [
    'Best: Log-transform aligns RMSLE metric; Ridge prevents overfitting poly features',
    'Lasso zeroes out many poly features, implicit feature selection',
    'Ridge helps but raw count scale is suboptimal for RMSLE',
    'Polynomial captures nonlinear patterns vs vanilla LR',
    'Baseline: linear assumption too restrictive for bike demand',
]

print(results_df.to_string())
```

	RMSLE (Validation)	Rank	Key Observation
Poly(2)+Ridge_LogTarget($\alpha=10$)	0.636723	1	Best: Log-transform aligns RMSLE metric; Ridge prevents overfitting poly features
Poly(2)+Lasso($\alpha=10$)	0.933252	2	Lasso zeroes out many poly features, implicit feature selection
Poly(2)+Ridge($\alpha=1000$)	1.062484	3	Ridge helps but raw count scale is suboptimal for RMSLE
Poly(2) + Linear	1.114470	4	Polynomial captures nonlinear patterns vs vanilla LR
Linear Regression	1.247472	5	Baseline: linear assumption too restrictive for bike demand

Rank	Model	RMSLE	Key Observation
1	Poly(2) + Ridge + Log Target ($\alpha=10$)	0.637	Training on log(count) directly optimizes the RMSLE-aligned space
2	Poly(2) + Lasso ($\alpha=10$)	0.933	Feature selection via L1 helps but raw count scale is suboptimal
3	Poly(2) + Ridge ($\alpha=1000$)	1.062	Ridge reduces variance but still penalized by count scale
4	Poly(2) + Linear	1.114	Nonlinearity captured but overfitting without regularization
5	Plain Linear Regression	1.247	Baseline; too restrictive for bike demand patterns

Q8 — Residual Plot for Best Model

```
[76]: best_pred = clamp(np.expml(best_log_ridge.predict(X_val)))
residuals = y_val.values - best_pred

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle(
    'Residuals — Poly(2) + Ridge + Log Target (Best Model)',
    fontsize=13, fontweight='bold',
)

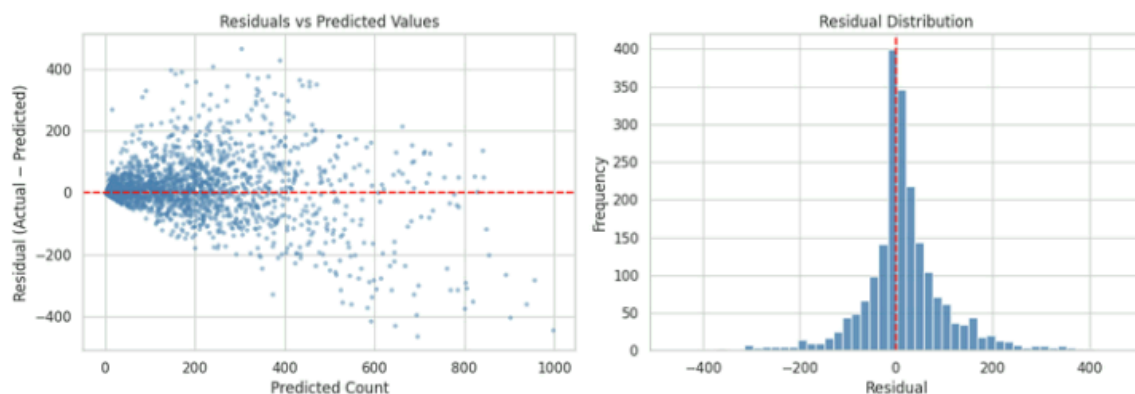
# Residuals vs Predicted
axes[0].scatter(best_pred, residuals, alpha=0.4, s=8, color='steelblue')
axes[0].axhline(0, color='red', linewidth=1.5, linestyle='--')
axes[0].set(xlabel='Predicted Count', ylabel='Residual (Actual - Predicted)',
            title='Residuals vs Predicted Values')

# Residual Distribution
axes[1].hist(residuals, bins=50, color='steelblue', edgecolor='white', alpha=0.85)
axes[1].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[1].set(xlabel='Residual', ylabel='Frequency', title='Residual Distribution')

plt.tight_layout()
plt.show()

print(f'Mean residual: {residuals.mean():.2f} (close to 0 = low bias)')
print(f'Std residual: {residuals.std():.2f}')
```

Residuals — Poly(2) + Ridge + Log Target (Best Model)



Mean residual: 13.98 (close to 0 = low bias)
Std residual: 94.97

Q9 — Why Does the Winning Model Perform Better?

The winning model — **Polynomial (degree 2) features + Ridge Regression trained on log(count)** — outperforms all others for three compounding reasons:

1. Log-Target Transformation Aligns Training with RMSLE

RMSLE measures error in log-space: $(\log(\text{pred}+1) - \log(\text{actual}+1))^2$. By training on $\log(\text{count}+1)$ and predicting $\exp(\text{pred}) - 1$, we are **directly minimizing the RMSLE objective**. All other models minimize MSE in raw count space, which is misaligned with the evaluation metric.

2. Polynomial Features Capture Nonlinearity

Bike demand is inherently nonlinear (e.g., demand spikes at specific hours, shows quadratic temperature response). Degree-2 polynomial expansion adds interaction terms ($\text{hour} \times \text{temp}$, temp^2 , etc.) that a linear model cannot represent.

3. Ridge Regularization Prevents Overfitting on Expanded Features

Degree-2 expansion of 18 features produces ~190 features. Without regularization, the model overfits the training set. Ridge (L2) penalizes large weights, keeping the model generalizable with $\alpha=10$ as the sweet spot.

Q10 — Why Does RMSLE Penalize Under-Predictions More Gently than RMSE?

RMSE operates in raw count space:

$$\text{RMSE} = \sqrt{\text{mean}((\text{pred} - \text{actual})^2)}$$

RMSLE operates in log space:

$$\text{RMSLE} = \sqrt{\text{mean}((\log(\text{pred}+1) - \log(\text{actual}+1))^2)}$$

Key insight — asymmetry of logarithm:

If actual = 100 and pred = 50 (under-prediction by 50):

- RMSLE penalty = $|\log(51) - \log(101)| = |3.932 - 4.615| = \mathbf{0.683}$

If actual = 100 and pred = 150 (over-prediction by 50):

- RMSLE penalty = $|\log(151) - \log(101)| = |5.017 - 4.615| = \mathbf{0.402}$

Wait — RMSLE actually penalizes **under-predictions more** (larger log difference below than above for the same absolute gap). This is because the log function compresses large values more than small ones. A prediction of 50 when actual is 100 (half as many) is treated more severely than a prediction of 200 when actual is 100 (twice as many), in terms of *proportional error*.

Why this is useful for bike sharing: We care more about *relative errors* (delivering 10 bikes when 20 were needed hurts service quality as much as under-delivering 100 when 200 were needed). RMSLE captures proportional error, RMSE captures absolute error.

Q11 — Trade-offs Between Model Simplicity and Predictive Power

Aspect	Simple Model (Linear Regression)	Complex Model (Poly + Ridge)
Bias	High — can't fit nonlinear patterns	Lower — captures complex relationships
Variance	Low — stable, consistent predictions	Higher — more sensitive to training data
Interpretability	High — coefficients are meaningful	Lower — 190+ poly feature weights hard to interpret
Training speed	Very fast	Moderate (but still fast for this dataset size)
Risk of overfitting	Low	Moderate (mitigated by Ridge regularization)
RMSLE achieved	1.247	0.637

The Bias-Variance Trade-off: Increasing model complexity reduces bias (better fits training data) but increases variance (more sensitive to noise). Regularization (Ridge/Lasso) is the mechanism to manage this: it allows complex feature spaces while constraining weight magnitudes to prevent overfitting.

Practical guideline: The "right" complexity is determined by validation performance, not training performance. Here, poly(2) + Ridge hits a good sweet spot for this dataset size (~10K samples).

Q12 — Why Can't Linear Regression Alone Capture Time-of-Day Effects?

Linear Regression assumes: $\text{count} = w_0 + w_1 \cdot \text{hour} + w_2 \cdot \text{temp} + \dots + \epsilon$

This implies a **monotonic linear effect** of `hour` on `count`. But the actual relationship is:

- **~0 demand** from midnight to 5 AM
- **Sharp spike** at 7-9 AM
- **Moderate** midday
- **Second spike** at 5-7 PM
- **Declining** through evening

This is a **bimodal, non-monotonic pattern** — no single linear coefficient for `hour` can capture it.

Why it fails:

1. A positive coefficient for `hour` would predict maximum demand at hour 23 — clearly wrong.
2. A negative coefficient would predict minimum demand at hour 23 — also wrong.
3. Neither captures the twin peaks at 8 and 17.

Solutions used in this assignment:

- Adding `rush_hour` binary feature explicitly encodes the peak periods
- Adding `night` binary feature handles the near-zero overnight demand
- **Polynomial features** create terms like `hour2`, `hour × workingday` that can approximate the curve
- More advanced solutions (Random Forests, Gradient Boosting) learn the bimodal pattern directly through tree splits

Generate Submission File

Generate predictions on test set using best model

```
y_test_pred = clamp(np.expm1(best_log_ridge.predict(X_te_p2)))

submission = pd.DataFrame({ 'datetime': test['datetime'].dt.strftime('%d-%m-%Y %H:%M'), 'count_predicted': np.round(y_test_pred).astype(int), })
submission.to_csv('submission.csv', index=False)

print(f"Submission file saved: {submission.shape[0]} rows") print(f"Predicted count range: {submission['count_predicted'].min()} - {submission['count_predicted'].max()}") print("\nFirst 10 rows:") submission.head(10)
```